

CSE-307 • OPERATING SYSTEMS

Term Paper (Part B) • Track 3

Deadlock Early Risk Classifier

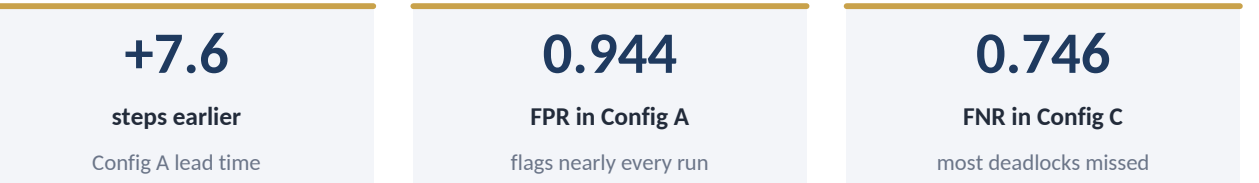
*Can a lightweight Decision Tree warn of deadlock
earlier than the Banker's Algorithm?*

Submitted by	Md. Nasimul Goni
Roll	202414095
Department	Computer Science & Engineering
Submitted to	Lec Khaled Hasan Irfan
Date	4 October 2026

Abstract

The Banker's Algorithm and resource-allocation-graph (RAG) cycle detection are exact, but they confirm that a state is unsafe only once it has been reached. This paper asks whether a lightweight Decision Tree (depth 4), looking only at the in-progress part of a request sequence, can flag a risky trajectory earlier than the Banker's safety check. The classifier is evaluated on three system sizes under a workload that changes character halfway through the trace, using average lead time, false-positive rate (FPR) and false-negative rate (FNR).

The result is negative but instructive. The tree warns earlier than Banker's only in the smallest system, and only by raising many false alarms. In larger systems it is late and misses most deadlocks, and its confidence does not track correctness. Banker's Algorithm remains the accurate and timely choice.



Contents

1. Problem Framing	2
2. Implementation Summary	2
3. Experimental Setup	2
4. Results	3
5. Analysis: What Changed Under the Shift, and Why	4
6. Conclusion	4
AI Disclosure	4

1. Problem Framing

The Banker's Algorithm and resource-allocation-graph (RAG) cycle detection are exact, but they only confirm that a state is unsafe once it has been reached. A real operating system would prefer a warning before that point. This project asks a narrow question: can a lightweight Decision Tree, looking only at the in-progress part of a request sequence, flag a risky trajectory earlier than the Banker's safety check does?

Success is measured by the **average lead time** (steps earlier or later than Banker's), the **false-positive rate** (FPR) and the **false-negative rate** (FNR), under a workload that changes character halfway through the trace.

2. Implementation Summary

- **Banker's Algorithm (ground truth).** Computes $\text{Need} = \text{Max} - \text{Allocation}$ and repeatedly finds a process whose need fits in the available resources; the state is safe if all processes can finish in some order. It also implements the request check (grant only if the resulting state is safe). Every state is labelled with this check, and unit tests reproduce the textbook example.
- **RAG cycle detection.** A depth-first search (white/grey/black colouring) over the process/resource graph. It is exact for single-instance resources; for multi-instance resources a cycle is only necessary, so Banker's stays the ground truth.
- **Learned component.** A scikit-learn `DecisionTreeClassifier` (`max_depth = 4`) trained on features computed only from the prefix of a sequence: utilization, the free fraction of the scarcest resource, request pressure and the largest per-resource pressure. The label of every prefix is whether its sequence eventually becomes unsafe. A shallow tree is cheap, interpretable and easy to explain in words. A unit test checks that features at step t do not change if later steps are removed (no leakage).
- **Workloads.** **RANDOM** (random valid requests, used for training) and **LATE_UNSAFE** (the shift): the first half only picks requests that keep the state safe, and from the halfway step on it picks requests that push the state unsafe.

3. Experimental Setup

A sequence has $4 \times (\text{number of processes})$ requests, applied one after another so the trajectory can be observed. Three sizes were used: **Config A** (4 processes, 2 resource types), **Config B** (8, 4) and **Config C** (12, 6). For each, the tree was trained on 1,000 **RANDOM** sequences and tested on 300 unseen sequences of each workload, so the **LATE_UNSAFE** test also acts as a distribution shift relative to training. The risk threshold is 0.5 and the random seed is fixed.

Lead time is the step at which Banker's marks the state unsafe minus the step at which the classifier first flags risk (positive = earlier), averaged over the unsafe sequences it flagged. **FPR** is the share of safe sequences flagged at any step, **FNR** the share of unsafe sequences never flagged, and the **early rate** the share flagged strictly before Banker's.

4. Results

Results on the shifted LATE_UNSAFE workload (the no-shift RANDOM results are in results/metrics_summary.csv in the repository):

Config	Procs × Res	Unsafe / 300	Avg lead time (steps)	FPR	FNR	Early rate	Conf. correct / wrong
A	4 × 2	48	+7.6	0.944	0.021	0.979	68.7 / 71.8
B	8 × 4	36	+1.6	0.617	0.444	0.472	56.4 / 68.3
C	12 × 6	59	-1.1	0.241	0.746	0.000	64.4 / 59.8

Table 1: Results on the LATE_UNSAFE workload (300 test sequences per configuration).

Banker's Algorithm blocks the unsafe request at the exact step where it appears, so its detection delay is zero by definition. The classifier's mean risk probability rose after the shift (0.64 → 0.70 in A, 0.48 → 0.54 in B, 0.36 → 0.41 in C), but only modestly. Its average lead time fell from +7.6 steps in Config A to +1.6 in B and -1.1 in C, i.e. it went from early to late as the system grew.

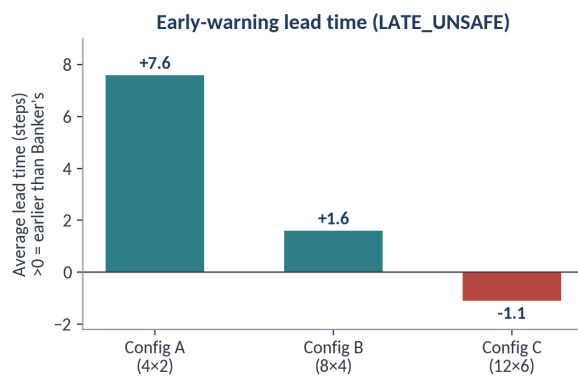


Fig. 1: Average lead time

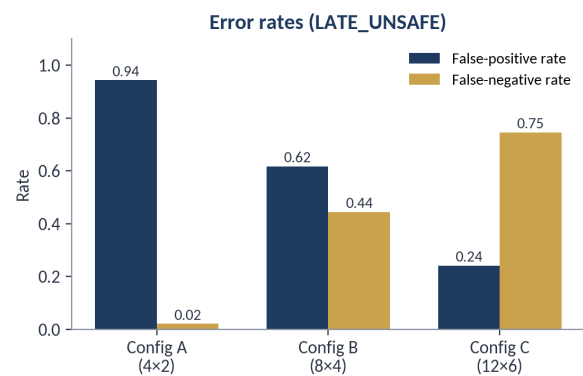


Fig. 2: False-positive / false-negative rates

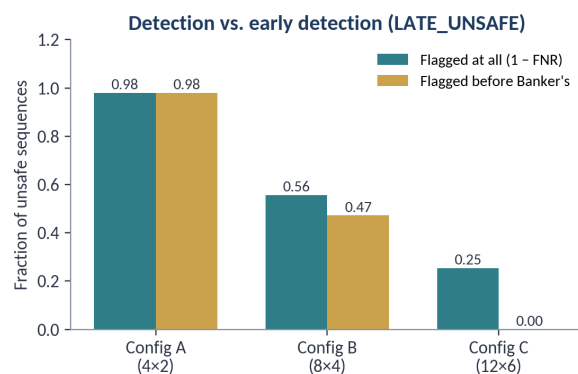


Fig. 3: Flagged at all vs. flagged before Banker's

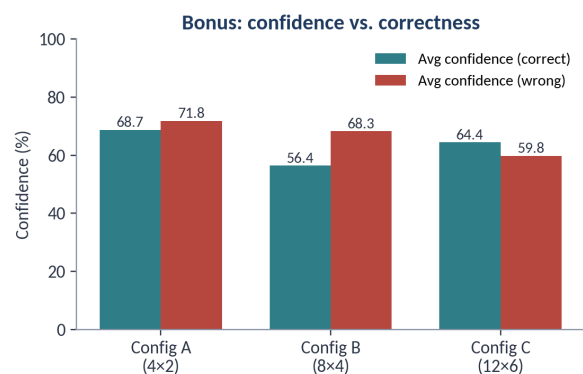


Fig. 4: Bonus – confidence, correct vs. wrong

5. Analysis: What Changed Under the Shift, and Why

Lead time shrinks as the system grows

The features are aggregates (utilization, pressure). In a larger system, one risky request changes them by a smaller proportion, so the classifier needs more steps before the change is visible; in Config C it becomes later than Banker's and flags none of the unsafe sequences early. Without any per-process information, such as how much each process still needs, it reacts to the shift rather than anticipating it.

Early warning is traded against false alarms

Config A looks best (lead +7.6, FNR 0.02), but its FPR is 0.94: it flags almost every sequence, so catching the unsafe ones is not impressive. Config C is the opposite, with a lower FPR (0.24) but an FNR of 0.75. No configuration achieves both low, which suggests the features cannot separate a busy but safe system from one heading towards deadlock.

Bonus track: explanation confidence

A rule-based template generator writes an explanation at each step (for example, “risk flagged: utilization is 83%, scarcest resource has 0% free”) next to Banker's verdict, saved in `results/explanations.txt`. The model's confidence was higher on wrong predictions than on correct ones in Configs A and B (71.8 vs 68.7 and 68.3 vs 56.4) and only slightly higher on correct ones in C (64.4 vs 59.8). So its confidence score is not a reliable indicator of correctness, and the model is not well calibrated.

6. Conclusion

A depth-4 Decision Tree on aggregate features cannot replace the Banker's Algorithm.

It can warn earlier in a small system, but only by raising many false alarms, and in larger systems it is late and misses most deadlocks. Its confidence does not track correctness. Banker's Algorithm remains the accurate and timely choice. Future work should add per-process features (such as how many processes can still finish with the available resources), tune the threshold, and train on trajectories closer to the shifted workload.

AI Disclosure

I used an AI assistant (Claude) to help write and structure the code in the repository (workload generator, experiment pipeline, plotting and unit tests), to debug it, and to polish the wording of this report. I ran the experiments, and all numbers in this report are the output of that code. I am responsible for understanding and explaining every part of the submission.